PROGRAMMING IN JAVA

ASSIGNMENT 2 – CO2

Strings, Collections Framework, Generics,
Comparators and Queue Design

Student details	Information
Name	Revi Thimma Reddy KC
Register Number	192325025
Course / Section	CSA0909-JAVA PROGRAMMING
Faculty Name	Mr. V surya
Submission Date	13-11-2026
GitHub Repository	https://github.com/ReviThimmaReddy/CSA0909-JAVA

Individual submission – 50 Marks

IMPORTANT SUBMISSION NOTE

This report is a structured solution and implementation guide for Q1–Q7. Replace the blank measurement cells with results from your own computer after compiling and running the included programs. Execution times depend on CPU, JVM, operating system, warm-up, heap settings and background load; therefore, no fabricated timing or memory measurements are presented.

You must personally execute every program, capture output screenshots, commit each task progressively to GitHub, and verify that the repository is accessible to the faculty. The report alone does not replace the required source files, raw measurements or GitHub history.

How to use this report

- Create a project using JDK 17 or later (or the exact version prescribed by your faculty).
- Create one folder per task: Task01 through Task07. Save each public class in a file with the same name.
- Compile with: javac FileName.java ; run with: java ClassName.
- Run benchmark programs several times. Use the average of repeated runs after warm-up, and record the JVM and machine details.
- Paste your real console screenshots and raw CSV data into the corresponding task folders.
- Replace the cover-page placeholders and add the final GitHub URL.

Contents

- Q1 – Building Text Under Load: String, StringBuilder or StringBuffer?
- Q2 – ArrayList, LinkedList and ArrayDeque
- Q3 – hashCode–equals Contract and HashSet
- Q4 – HashMap, TreeMap and LinkedHashMap
- Q5 – Generic Leaderboard and Comparator
- Q6 – Priority Queue and Fair Hospital Triage
- Q7 – Diagnose, Redesign and Validate
- GitHub, README and evidence checklist

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer? (6 Marks)

1.1 Prediction and principle

String is immutable. An expression such as text = text + field creates a new String for the combined content; the previous value cannot be modified. Repeating this in a loop can repeatedly copy the already-built prefix. StringBuilder maintains a mutable character buffer and appends into it, expanding capacity occasionally. StringBuffer provides similar mutable operations but synchronizes its methods, which adds overhead.

Approach	Prediction as fields increase	Thread behaviour
String + in a loop	Highest allocation and copying cost for long output; potentially quadratic total character copying.	Safe to share as an immutable value, but repeated concatenation is not an efficient shared mutable log.
StringBuilder	Usually fastest for a single thread; amortized linear append work.	Not thread-safe. Use one builder per ticket or synchronize externally for a shared log.
StringBuffer	Similar growth strategy to StringBuilder, with synchronization overhead.	Individual methods are synchronized; compound multi-call logic may still need external coordination.

1.2 Three designed workloads

Workload	Design	Purpose
A – Small	1 ticket × 6 fields; 20 repetitions	Shows whether differences are negligible for a short ticket.
B – Large	10,000 tickets × 50 fields	Exposes repeated copying and allocation at scale.
C – Stress	2,000 tickets × 250 fields, with each field around 40 characters	Tests long output and buffer growth.

Prediction: workload A may show little practical difference because fixed overhead and timer noise dominate. Workloads B and C should make repeated String concatenation substantially more expensive. StringBuilder should generally be faster than StringBuffer in a single-thread benchmark because it avoids synchronization.

1.3 Implementation

```
import java.util.*;

public class TicketBenchmark {
    static String[] fields(int count) {
        String[] a = new String[count];
        for (int i = 0; i < count; i++)
            a[i] = "FIELD-" + i + "-PASSENGER-DETAIL";
        return a;
```

```
  }

  static String usingString(String[] a) {
     String result = "";
     for (String s : a) result = result + s + "|";
     return result;
  }

  static String usingBuilder(String[] a) {
     StringBuilder b = new StringBuilder();
     for (String s : a) b.append(s).append('|');
     return b.toString();
  }

  static String usingBuffer(String[] a) {
     StringBuffer b = new StringBuffer();
     for (String s : a) b.append(s).append('|');
     return b.toString();
  }

  static long measure(Runnable task) {
     long start = System.nanoTime();
     task.run();
     return System.nanoTime() - start;
  }

  public static void main(String[] args) {
     int[][] workloads = {{1, 6}, {10000, 50}, {2000, 250}};
     for (int[] w : workloads) {
        String[] a = fields(w[1]);
        // Warm-up
        for (int i = 0; i < 3; i++) {
           usingString(a); usingBuilder(a); usingBuffer(a);
        }
        long s = 0, b = 0, f = 0;
        int repeats = 5;
        for (int r = 0; r < repeats; r++) {
           s += measure(() -> { for(int i=0;i<w[0];i++) usingString(a); });
           b += measure(() -> { for(int i=0;i<w[0];i++) usingBuilder(a); });
           f += measure(() -> { for(int i=0;i<w[0];i++) usingBuffer(a); });
        }
        System.out.printf("tickets=%d fields=%d String=%.3f ms Builder=%.3f ms Buffer=%.3f
ms%n",
```

```
            w[0], w[1], s/5_000_000.0, b/5_000_000.0, f/5_000_000.0);
        }
    }
}
```

The benchmark reports elapsed time for each approach across five measured repetitions, with a small warm-up. For a more rigorous study, rotate the execution order between repetitions and run the program in an otherwise idle environment. Use a profiler or Java Flight Recorder to measure allocation and memory accurately; Runtime.freeMemory() snapshots alone are noisy and are not a reliable count of allocated objects.

1.4 Measurement record

Workload	String avg (ms)	StringBuilder avg (ms)	StringBuffer avg (ms)	Observed winner / notes
A – 1 × 6	_____	_____	_____	_____
B – 10,000 × 50	_____	_____	_____	_____
C – 2,000 × 250	_____	_____	_____	_____

1.5 Object creation and complexity

Let n be the number of fields and L be the final number of characters. Repeated concatenation copies progressively larger prefixes. With similarly sized fields, total copied characters can grow as $O(n^2)$ (more precisely, the sum of prefix lengths). StringBuilder appends each character into a growable buffer, giving O(L) total character work, commonly described as O(n) when field lengths are bounded. StringBuffer has the same asymptotic append complexity but synchronization overhead.

Intermediate String count for the + loop is implementation-dependent because the compiler and JVM may optimize some expressions. In a loop, each reassignment normally creates a new combined String, so use approximately one new result String per iteration as a conceptual estimate, not an exact JVM allocation count. Builder and Buffer create a mutable buffer, may allocate replacement arrays during growth, and create one final String when toString() is called.

1.6 Design decision and alternative

Use a fresh StringBuilder for each ticket: ticket construction is local to one request and does not require synchronization. For a shared booking log, prefer a dedicated logging mechanism or a synchronized/locked append operation around a shared StringBuilder. If a simple shared mutable character buffer is explicitly required, StringBuffer is a defensible choice for synchronized individual operations, but ensure the whole logical append sequence is atomic when necessary.

String.join(delimiter, fields) is concise when the fields are already available as strings; it joins them into one result. StringJoiner is useful when constructing a delimited sequence incrementally and supports prefix/suffix. String.format is expressive for formatted output but generally incurs formatting overhead and is not the preferred high-throughput choice. For this ticket workload, a pre-sized StringBuilder is the recommended implementation.

```
String ticket = String.join(" | ", fields);

StringJoiner joiner = new StringJoiner(" | ", "TICKET[", "]");
```

```
for (String field : fields) joiner.add(field);
String ticket2 = joiner.toString();

StringBuilder b = new StringBuilder(estimatedCharacters);
for (String field : fields) b.append(field).append(" | ");
String ticket3 = b.toString();
```

Conclusion: confirm the prediction using the completed measurement table. Do not claim a specific speed-up until the values have been measured on the submission machine.

Q2 – When Is ArrayList an Appropriate Engineering Choice? (4 Marks)

2.1 Analysis

ArrayList stores references in a resizable array. It offers constant-time indexed access and amortized constant-time append, but insertion/removal near the front or middle shifts subsequent references. LinkedList is a doubly linked list: insertion/removal at a known iterator position is constant time, but locating an index requires traversal. ArrayDeque is a resizable circular array designed for efficient operations at both ends; it is suitable for an Up Next queue but does not support indexed access.

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	O(1) amortized	O(1)	O(1) amortized at tail
Insert at front	O(n) shifts	O(1)	O(1) amortized
Insert in middle	O(n) search/shift	O(n) locate + O(1) link	Not supported as indexed insertion
Get k-th	O(1)	O(n)	Not supported
Iterate all	O(n), cache-friendly	O(n), pointer traversal	O(n)
Remove front	O(n) shifts	O(1)	O(1) amortized

ArrayList uses a backing array of references (unused capacity may be reserved). LinkedList stores a node per element with item, previous and next references, so its per-element overhead is higher. ArrayDeque uses an array of references and circular indices.

2.2 Experiment plan and implementation

Use playlist sizes 1,000, 10,000 and 100,000. For fair tests, prepare equivalent lists before timing each operation. Do not repeatedly measure a destructive operation on an already depleted collection. Use System.nanoTime(), warm-up, at least five repetitions and report averages.

```
import java.util.*;

public class PlaylistBenchmark {
   static List<Integer> array(int n) {
      List<Integer> x = new ArrayList<>();
      for (int i=0;i<n;i++) x.add(i);
      return x;
   }
   static List<Integer> linked(int n) {
      List<Integer> x = new LinkedList<>();
      for (int i=0;i<n;i++) x.add(i);
      return x;
   }
   static long time(Runnable r) {
      long s=System.nanoTime(); r.run(); return System.nanoTime()-s;
   }
   public static void main(String[] args) {
      int[] sizes={1000,10000,100000};
```

```
        for(int n:sizes) {
            List<Integer> a=array(n), l=linked(n);
            long ta=time(()->{for(int i=0;i<20000;i++) a.get(n/2);});
            long tl=time(()->{for(int i=0;i<20000;i++) l.get(n/2);});
            long ia=time(()->{for(int i=0;i<1000;i++) a.add(0,-i);});
            long il=time(()->{for(int i=0;i<1000;i++) l.add(0,-i);});
            long xa=time(()->{long sum=0; for(int v:a) sum+=v;});
            long xl=time(()->{long sum=0; for(int v:l) sum+=v;});
            System.out.printf("n=%d get-middle A=%d ns L=%d ns add-front A=%d ns L=%d ns iterate
A=%d ns L=%d ns%n",
                n,ta,tl,ia,il,xa,xl);
        }
        Deque<Integer> upNext = new ArrayDeque<>();
        upNext.addFirst(10); upNext.addLast(20);
        System.out.println("Next song: " + upNext.removeFirst());
    }
}
```

2.3 Measurement table

n	Operation	ArrayList (ns)	LinkedList (ns)	Interpretation
1,000	get middle × 20,000	_____	_____	_____
1,000	add front × 1,000	_____	_____	_____
1,000	full iteration	_____	_____	_____
10,000	get middle × 20,000	_____	_____	_____
10,000	add front × 1,000	_____	_____	_____
10,000	full iteration	_____	_____	_____
100,000	get middle × 20,000	_____	_____	_____
100,000	add front × 1,000	_____	_____	_____
100,000	full iteration	_____	_____	_____

2.4 Engineering recommendation

ArrayList is a reasonable choice when users mostly browse, jump to a song by index, iterate, and append occasionally. It becomes less attractive when frequent insertions/removals occur near the front or middle and the list is large. Even then, LinkedList is not automatically faster: finding the middle position remains O(n), and its node overhead and cache behaviour can make iteration slower.

For the changed Up Next requirement, use ArrayDeque because both-end insertion and front removal are amortized O(1), with compact array storage. Keep the main editable playlist as ArrayList if random access is important; use a separate ArrayDeque for the queue of upcoming songs.

2.5 Plain-array alternative

A manually managed array would store Song references in contiguous slots. Appending is O(1) if capacity remains; insertion/removal at index i requires shifting O(n-i) references; get(k) is O(1); iteration is O(n). It can reduce framework overhead but requires manual resizing, bounds checks and shifting logic. ArrayList provides this design with safer, maintained resizing.

Q3 – Engineer the hashCode–equals Contract (6 Marks)

3.1 Contract and prediction

The equals/hashCode contract requires that equal objects have equal hash codes. If equals() returns true for two Student objects, their hashCode() values must be identical. Unequal objects may share a hash code; HashSet resolves collisions by checking equality. If equals() is overridden but hashCode() is not, Object's identity-based hash code may send logically equal students to different buckets.

Test	Input design	Expected size before run (broken class)	Reason
1 – Apparent success	Insert the exact same object reference twice	1	The same object has the same identity hash and is equal to itself.
2 – Expose defect	Insert two different Student objects with same regNo	Usually 2	equals says equal, but inherited identity hash codes can route them to different buckets.
3 – Mutation test	Insert a student, then change a mutable regNo used by equals/hashCode	Lookup/removal may fail	The object may remain in the bucket selected by its old hash.

The precise result of test 2 is technically dependent on hash bucket mapping and collisions. To make the experiment deterministic, print both identity hash codes and repeat with multiple distinct duplicate instances; the broken implementation violates the contract and is not reliable.

3.2 Complete-the-code program

```
import java.util.HashSet;
import java.util.Set;

class Student {
   private final String regNo;
   private final String name;
   Student(String regNo, String name) {
      this.regNo = regNo; this.name = name;
   }
   @Override public boolean equals(Object o) {
      if (this == o) return true;
      if (!(o instanceof Student)) return false;
      Student other = (Student)o;
      return regNo.equals(other.regNo);
   }
   @Override public int hashCode() {
      return regNo.hashCode();
   }
   @Override public String toString() { return regNo + " " + name; }
```

```
}
public class Attendance {
   public static void main(String[] args) {
      Set<Student> present = new HashSet<>();
      present.add(new Student("192311001", "Arun"));
      present.add(new Student("192311002", "Divya"));
      present.add(new Student("192311001", "Arun"));
      System.out.println("Unique students: " + present.size());
   }
}
```

Correct output: Unique students: 2. Before TODO 1, record the actual output produced by your JDK. The broken version may report 3 because the two equal objects can be placed in different buckets. After implementing hashCode() from regNo, both objects have the same hash and HashSet can apply equals() to reject the duplicate.

3.3 HashSet behaviour and mutation

HashSet uses hashCode() to determine a bucket, then uses equals() among candidate entries in that bucket. If a field contributing to hashCode() changes after insertion, the object is still stored in the old bucket while future lookup calculates a new bucket. This can make contains() or remove() fail. The safest design is to make equality-key fields final and immutable, as in the completed Student class.

3.4 Alternative structures and complexity

Structure	De-duplication key	Per scan	Space for n unique records	Notes
HashSet<Student>	Student.equals/hashCode	O(1) average	O(n)	Requires contract-correct immutable key fields.
TreeSet<Student>	Comparator on regNo	O(log n)	O(n)	Comparator must return 0 exactly for IDs treated as duplicates.
HashMap<String,Student>	regNo string key	O(1) average	O(n)	Use putIfAbsent(regNo, student) to preserve first scan.

```
Map<String, Student> byRegNo = new HashMap<>();
byRegNo.putIfAbsent(student.regNo, student);

// Alternative: ordered de-duplication
Set<Student> ordered = new TreeSet<>(
   Comparator.comparing(s -> s.regNo)
);
```

Verification should include: two different objects with the same registration number; two different registration numbers; contains() on a newly created equal object; and removal using a newly created equal object. Confirm that the set size remains 2 for the supplied attendance data.

Q4 – Choosing a Map for a Pharmacy Inventory (6 Marks)

4.1 Map comparison and prediction

Map	put/get	Iteration order	Natural report use
HashMap	O(1) average; worst-case can degrade	No guaranteed order	Fast code lookup; sort entries separately for code report.
TreeMap	O(log n)	Sorted by key	Direct code-order report.
LinkedHashMap	O(1) average	Insertion order by default	Direct added-order report.

HashMap and LinkedHashMap provide expected constant-time key operations when hashes are well distributed. TreeMap is a balanced search tree and maintains key order. Iterating any map is O(n) in the number of entries; producing a sorted report from HashMap adds O(n log n) sorting.

4.2 Implementation

```
import java.util.*;

public class PharmacyInventory {
   static Map<String,Integer> sample(Map<String,Integer> map, int n) {
      for (int i=0;i<n;i++) {
         String code=String.format("MED%06d", (n-i)*3);
         map.put(code, 100+i);
      }
      return map;
   }
   static long lookups(Map<String,Integer> map, List<String> keys) {
      long start=System.nanoTime();
      long sum=0;
      for(int repeat=0;repeat<100000;repeat++)
         sum += map.getOrDefault(keys.get(repeat%keys.size()),0);
      long elapsed=System.nanoTime()-start;
      if(sum==-1) System.out.println(sum);
      return elapsed;
   }
   public static void main(String[] args) {
      for(int n:new int[]{1000,10000,100000}) {
         Map<String,Integer> h=new HashMap<>();
         Map<String,Integer> t=new TreeMap<>();
         Map<String,Integer> l=new LinkedHashMap<>();
         sample(h,n); sample(t,n); sample(l,n);
         List<String> keys=new ArrayList<>(h.keySet());
         System.out.printf("n=%d lookup(ns): HashMap=%d TreeMap=%d
LinkedHashMap=%d%n",
```

```
            n,lookups(h,keys),lookups(t,keys),lookups(l,keys));
        long s=System.nanoTime(); h.entrySet().stream()
            .sorted(Map.Entry.comparingByKey()).count();
        long hashSort=System.nanoTime()-s;
        s=System.nanoTime(); t.entrySet().forEach(e -> {});
        long treeReport=System.nanoTime()-s;
        s=System.nanoTime(); l.entrySet().forEach(e -> {});
        long insertionReport=System.nanoTime()-s;
        System.out.printf("reports(ns): HashMap-sort=%d TreeMap-code-order=%d
LinkedHashMap-insertion-order=%d%n",
            hashSort,treeReport,insertionReport);
      }
    }
}
```

The sample program measures 100,000 lookups and report traversal/sorting for 1,000, 10,000 and 100,000 medicines. For robust evidence, repeat each measurement at least five times and report the average. A production benchmark should use pre-generated keys, include warm-up, and consume lookup results so the work cannot be optimized away.

4.3 Measurement and acceptability

Medicines	Map	100,000 lookups avg (ms)	Code-order report (ms)	Insertion-order report (ms)
1,000	HashMap	_____	_____	_____
1,000	TreeMap	_____	_____	_____
1,000	LinkedHashMap	_____	_____	_____
10,000	HashMap	_____	_____	_____
10,000	TreeMap	_____	_____	_____
10,000	LinkedHashMap	_____	_____	_____
100,000	HashMap	_____	_____	_____
100,000	TreeMap	_____	_____	_____
100,000	LinkedHashMap	_____	_____	_____

Define the service target before selecting a map. Example criterion: a single in-memory stock lookup should complete within 1 ms under the expected load on the deployment server. This is a proposed engineering target, not a measured result; revise it to match the hospital's service-level requirement. The remaining lookup cost includes hashing/comparison, memory access, possible collision handling, and JVM/runtime overhead.

A practical design is often a HashMap for primary code lookups plus a report view generated on demand, or a LinkedHashMap if insertion-order reporting is frequent and important. If code-order reporting must be immediate and frequent, TreeMap is a straightforward single-structure solution. If both orders are critical, maintain two indexes or sort a snapshot for one report, accounting for the extra memory and update work.

4.4 Why not Hashtable? Array alternative

Hashtable is a legacy synchronized map that does not allow null keys or values. Modern code generally prefers HashMap, LinkedHashMap or TreeMap; when concurrency is required, ConcurrentHashMap is usually a better choice than synchronizing every Hashtable operation.

A sorted array of medicine records supports binary search in O(log n) lookup and O(n) insertion because elements may need shifting. Code-order report generation is O(n) because the array is already sorted. Its storage is O(n) and can be compact, but updates are expensive compared with map insertion. HashMap offers O(1) average lookup and update but needs extra table capacity; TreeMap offers O(log n) lookup/update and keeps order in a node-based structure.

Q5 – Derive a Generic Ranking Component from a Principle (7 Marks)

5.1 Design reasoning

Use a separate Comparator<Team> rather than hard-coding Team's natural order when the ranking policy is specific to a competition or may change. A reusable Leaderboard<T> accepts Comparator<? super T>, which means it can compare T values using a comparator defined for T or one of its supertypes. This is flexible for teams, players, colleges and quiz groups.

The ranking order is lexicographic: points descending; if tied, goal difference descending; if still tied, name ascending. Each tie-breaker is consulted only when all earlier comparison keys are equal. Do not compare integers by subtraction because overflow can produce an incorrect sign.

5.2 Completed program

```
import java.util.*;

class Team {
   final String name;
   final int points;
   final int goalDiff;
   Team(String name, int points, int goalDiff) {
      this.name=name; this.points=points; this.goalDiff=goalDiff;
   }
   @Override public String toString() {
      return name+"("+points+","+goalDiff+")";
   }
}

class Leaderboard<T> {
   private final List<T> items = new ArrayList<>();
   private final Comparator<? super T> order;
   Leaderboard(Comparator<? super T> order) { this.order=order; }
   void add(T item) { items.add(item); }
   List<T> ranking() {
      List<T> copy = new ArrayList<>(items);
      copy.sort(order);
      return copy;
   }
}

public class LeaderboardDemo {
   public static void main(String[] args) {
      Comparator<Team> byRank =
         Comparator.comparingInt((Team x) -> x.points).reversed()
            .thenComparing(
               Comparator.comparingInt((Team x) -> x.goalDiff).reversed())
```

```
            .thenComparing(x -> x.name);

        Leaderboard<Team> board = new Leaderboard<>(byRank);
        board.add(new Team("Panthers",9,4));
        board.add(new Team("Eagles",9,6));
        board.add(new Team("Cobras",6,1));
        board.add(new Team("Bisons",9,4));
        System.out.println(board.ranking());
    }
}
```

Expected output: [Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]. Bisons ranks above Panthers because points and goal difference are equal, so the final alphabetical comparison places 'Bisons' before 'Panthers'. ranking() sorts a copy, leaving the original insertion list unchanged.

5.3 Comparator correctness and adversarial tests

The comparator is transitive because it compares a lexicographic tuple of well-defined orders: negative points order, negative goal-difference order, and normal String name order. If A precedes B and B precedes C under the first differing key, A also precedes C. If all three keys are equal, compare returns zero. The final name key also makes distinct named teams deterministic.

- Tie test: three teams with equal points but different goal differences.
- Second tie test: equal points and goal difference, with names inserted in reverse alphabetical order.
- Faulty comparator test: compare points using a.points - b.points. Large positive/negative values can overflow; use Integer.compare or comparingInt instead.
- Faulty tie-breaker test: compare only points. Teams with equal points then compare as equal even when their goal differences and names differ.

5.4 Alternatives and complexity

Approach	Time	Extra space	Reuse
Comparator chain + one sort	O(n log n)	O(n) copy for non-mutating ranking; sort implementation uses temporary workspace	High: each type can supply a different comparator.
Comparable<Team>	O(n log n)	O(n) copy if original must be preserved	Natural order is tied to Team and less flexible for alternate ranking policies.
Three stable sorts: name, then goalDiff, then points	3 × O(n log n) = O(n log n)	Depends on sorting/copy strategy; potentially extra list storage	Works only if each sort is stable and performed from least to most significant key.

Collections.sort(list, comparator) and list.sort(comparator) both sort according to the comparator. The generic method can also accept List<? extends T> as a source when copying values into a List<T>; this wildcard is useful for accepting a list of T or a subtype of T without allowing unsafe additions to the source list.

Q6 – Priority, Order and Fairness in a Queue (8 Marks)

6.1 Design

A FIFO Queue treats the earliest arrival as next regardless of severity, so it cannot enforce triage priority. A PriorityQueue orders elements according to a comparator and allows poll() to remove the highest-priority element. Since Java PriorityQueue is not stable for equal priorities, add an arrival sequence number as a secondary comparison key.

6.2 Complete implementation

```
import java.util.*;

class Patient {
   final String name;
   final int severity;
   final long arrival;
   Patient(String name,int severity,long arrival) {
      this.name=name; this.severity=severity; this.arrival=arrival;
   }
   @Override public String toString() { return name+"(S"+severity+")"; }
}

public class Triage {
   private long counter=0;
   private final PriorityQueue<Patient> queue = new PriorityQueue<>(
      Comparator.comparingInt((Patient p)->p.severity)
            .thenComparingLong(p->p.arrival)
   );
   void admit(String name,int severity) {
      if(severity<1 || severity>5)
         throw new IllegalArgumentException("Severity must be 1 to 5");
      queue.add(new Patient(name,severity,counter++));
   }
   Patient next() { return queue.poll(); }
   public static void main(String[] args) {
      Triage er=new Triage();
      er.admit("Ravi",3); er.admit("Meena",1); er.admit("John",3);
      er.admit("Asha",5); er.admit("Kumar",1); er.admit("Lata",2);
      StringBuilder order=new StringBuilder();
      Patient p;
      while((p=er.next())!=null) order.append(p).append(' ');
      System.out.println(order.toString().trim());
   }
}
```

Expected treatment order: Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5). The counter increases on every admission. When severity is equal, the smaller arrival value is compared first, preserving FIFO order within that severity.

Printing a PriorityQueue directly does not guarantee sorted treatment order. Its iterator exposes the heap's internal arrangement, not a sorted traversal. Repeatedly call poll() on a copy to display treatment order without destroying the live queue.

6.3 Complexity and experiment

Design	Admit	Serve next	Space	Comment
PriorityQueue	O(log n)	O(log n)	O(n)	Heap; peek is O(1).
Sorted LinkedList	O(n) search + O(1) link	O(1) at head	O(n) nodes	Maintaining sorted order makes admission linear.
Five FIFO queues	O(1) per severity queue	O(1) to find first non-empty severity (five checks)	O(n)	Excellent when severity domain is fixed at 1–5.

Benchmark at 100, 1,000, 10,000 and 100,000 waiting patients. Generate deterministic severity and arrival values, time batches of admissions and removals, and average at least five runs. Keep input generation outside the timed region.

```
import java.util.*;
public class QueueGrowth {
   static long run(int n) {
      PriorityQueue<Integer> q = new PriorityQueue<>();
      long start=System.nanoTime();
      for(int i=0;i<n;i++) q.add((i*37)%5);
      while(!q.isEmpty()) q.poll();
      return System.nanoTime()-start;
   }
   public static void main(String[] args) {
      for(int n:new int[]{100,1000,10000,100000}) {
         for(int w=0;w<2;w++) run(n); // warm-up
         long total=0;
         for(int r=0;r<5;r++) total+=run(n);
         System.out.printf("n=%d average=%.3f ms%n",n,total/5_000_000.0);
      }
   }
}
```

Queue size	Average batch time (ms)	Notes
100	_____	_____

1,000	_____	_____
10,000	_____	_____
100,000	_____	_____

6.4 Starvation and deployment

A severity-5 patient can wait indefinitely if a continuous stream of severity-1 to severity-4 patients always arrives first. This is starvation. One mitigation is aging: increase effective priority as waiting time grows, while preserving emergency safety rules. For example, every fixed waiting interval can reduce the effective numeric severity by one level, down to a configured maximum priority. Recompute priorities periodically or use a scheduling policy that calculates effective priority at selection time.

A city hospital receiving 5,000 patients per day averages about 0.058 patients per second over 24 hours, although real arrivals are bursty. A binary heap handles 5,000 daily admissions comfortably in algorithmic terms: admission and removal are O(log n), and memory is O(n). Deployment suitability still depends on peak concurrent queue size, persistence, multiple triage stations, clinical escalation policy, auditability and failover. The data structure alone does not establish clinical safety.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected limitation: repeated String concatenation in ticket generation

This redesign is based on Q1 because repeated concatenation has a clear scaling limitation and can be measured with controlled workloads. The original solution repeatedly creates a new immutable String while copying the previous ticket prefix.

A. Diagnose the bottleneck (2 marks)

For n similarly sized fields, the original loop copies a growing prefix on each iteration. The sum of copied prefix lengths grows approximately as $1 + 2 + \ldots + n$, giving $O(n^2)$ character-copy work. The dominant costs at high field counts are repeated copying and intermediate String allocation. The evidence must be taken from the Q1 benchmark: insert the measured time and allocation observations below.

Evidence item	Your Q1 observation
Largest workload tested	_____
String average time	_____
StringBuilder average time	_____
Observed scaling / allocation evidence	_____

B. Define the design objective (2 marks)

- Reduce ticket construction from repeated prefix copying toward O(L), where L is the final ticket length.
- Reduce intermediate String creation to one final immutable String per ticket, apart from input field objects.
- Preserve the exact ticket text and field order.
- Set an experimental target: for the largest workload, the redesigned implementation should be faster than the original in repeated averages; report the actual percentage only after measuring.

C. Revised design (4 marks)

Use a local StringBuilder with an estimated initial capacity. The builder's mutable character buffer allows each field to be appended without reconstructing the entire prefix. Capacity estimation reduces buffer growth and replacement-array copying. The builder is local to one ticket, so no synchronization is needed.

```
static String buildTicket(String[] fields) {
    int capacity = 0;
    for (String field : fields) {
        if (field != null) capacity += field.length();
        capacity += 2; // delimiter and separator allowance
    }
    StringBuilder ticket = new StringBuilder(capacity);
    for (int i=0; i<fields.length; i++) {
        if (i > 0) ticket.append('\n');
        ticket.append(fields[i]);
```

```
    }
    return ticket.toString();
}
```

Design flow: input fields → estimate output capacity → append fields and delimiters to local builder → convert once to String → return immutable ticket. The trade-off is a small pre-pass over the fields to estimate capacity. This adds O(n) work but can avoid repeated capacity expansion. The output String still requires O(L) storage.

D. Predict before implementing (1 mark)

Prediction: the redesigned solution should produce identical ticket content while reducing construction time and temporary allocation for large tickets. The improvement should be most visible for many fields and long strings; for a tiny ticket, the extra capacity-estimation pass may make little difference.

E. Implement and validate (2 marks)

```
static String original(String[] fields) {
    String result = "";
    for (String field : fields) result = result + field + "\n";
    return result;
}
static String redesigned(String[] fields) {
    int capacity = 0;
    for (String field : fields) capacity += (field == null ? 4 : field.length()+1);
    StringBuilder b = new StringBuilder(capacity);
    for (String field : fields) b.append(field).append('\n');
    return b.toString();
}
```

Workload	Original avg (ms)	Redesigned avg (ms)	Original output equals redesigned?	Improvement %
1 × 6	_____	_____	Yes / No	_____
10,000 × 50	_____	_____	Yes / No	_____
2,000 × 250	_____	_____	Yes / No	_____

Calculate improvement percentage as ((original average − redesigned average) / original average) × 100. Use identical field arrays and ticket counts for both implementations. Include warm-up and repeated runs. Validate output equality with equals() before accepting performance results.

F. Defend the final design (2 marks)

If the measurements show lower time and allocation for the large workloads, the prediction is supported. The redesign pays a small O(n) capacity-estimation pass and allocates a builder buffer plus the final String, but avoids repeated intermediate concatenated Strings. For a tiny one-off expression, direct + is clearer and the performance difference may be negligible. For a shared concurrent log, this local-builder design is not itself a shared logging solution; use a proper thread-safe logging facility or explicit synchronization around shared state.

Q7 conclusion

The revised design is preferable for high-volume ticket generation because it directly removes repeated prefix copying while preserving output. The final conclusion must cite your actual benchmark table and state the observed improvement, not a guessed value.

GitHub Repository and README Requirements

Repository name: CSA09-<RegisterNumber>-Assignment2

Recommended folder structure

```
CSA09-<RegisterNumber>-Assignment2/
│
├── README.md
├── Task01/
│   ├── TicketBenchmark.java
│   ├── raw-results.csv
│   └── screenshots/
├── Task02/
│   ├── PlaylistBenchmark.java
│   └── screenshots/
├── Task03/
│   ├── Attendance.java
│   └── screenshots/
├── Task04/
│   ├── PharmacyInventory.java
│   └── screenshots/
├── Task05/
│   ├── LeaderboardDemo.java
│   └── screenshots/
├── Task06/
│   ├── Triage.java
│   ├── QueueGrowth.java
│   └── screenshots/
└── Task07/
    ├── TicketRedesign.java
    ├── comparison.csv
    └── screenshots/
```

README.md template

```
# CSA09 – Programming in Java – Assignment 2

- Name: YOUR NAME
- Register Number: YOUR REGISTER NUMBER
- Java version: (paste `java -version`)
- Repository: CSA09-<RegisterNumber>-Assignment2

## Compile and run
Open a terminal in the required task folder.

Example:
```

```bash
javac Attendance.java
java Attendance
```

Repeat the commands for each public Java class. Keep each public class in a file with the same name.

Task results
- Task01: Compare String, StringBuilder and StringBuffer using repeated runs.
- Task02: Compare ArrayList, LinkedList and ArrayDeque operations.
- Task03: Verify HashSet de-duplication using equals/hashCode.
- Task04: Compare HashMap, TreeMap and LinkedHashMap.
- Task05: Verify generic leaderboard ordering.
- Task06: Verify severity and arrival-order triage.
- Task07: Compare original and redesigned ticket construction.

Evidence
Raw benchmark data, console output screenshots and analysis are stored in the corresponding Task folders.

Progressive Git commits

```
git init
git add README.md
git commit -m "Add assignment README"

git add Task01
git commit -m "Complete Task 1 string benchmark"

git add Task02
git commit -m "Complete Task 2 collection comparison"

# Repeat one task at a time through Task07
git remote add origin https://github.com/<username>/CSA09-<RegisterNumber>-Assignment2.git
git branch -M main
git push -u origin main
```